

Playwright Capstone Project

AJIO E-Commerce Automation Testing Report

Field	Details
Author	Shiny Belsiya R
Application Under Test	AJIO - https://www.ajio.com
Automation Framework	Playwright with JavaScript and Page Object Model
Planned Coverage	9 service modules, 135 planned scenarios

Project Overview

This capstone report defines a professional automation testing plan for the AJIO e-commerce website using Playwright with JavaScript. The project is planned as a QA automation framework, not as a website development activity. The main intention is to validate customer-facing workflows such as navigation, product discovery, product detail validation, shopping bag handling, promotions, wishlist behavior, API/network observation, responsive layouts, and regression evidence generation.

The framework follows the Page Object Model so that test scripts remain readable, reusable, and easier to maintain when the live website changes. Since AJIO is a real commercial website, the plan avoids unsafe actions such as completing payment, using real customer information, or depending on restricted backend access. Instead, the project focuses on stable automation that can be executed repeatedly with dummy data and safe validations.

Website Selection Rationale

AJIO was selected because it provides a real shopping experience with dynamic product listings, category navigation, offers, product detail pages, wishlist entry points, bag/cart behavior, and responsive frontend design. These characteristics make it suitable for demonstrating frontend automation, dynamic locator strategy, network monitoring, and user journey validation in a realistic environment.

- The website represents a live e-commerce domain with realistic catalogue and product browsing workflows.
- It supports meaningful automation coverage without requiring database access or application source code.
- It contains dynamic UI behavior, so the project can demonstrate robust Playwright locators and assertions.
- It allows safe validation of product display, cart/bag behavior, pricing labels, promotions, and page navigation.
- It is suitable for evidence generation through screenshots, traces, execution reports, and CI workflow results.

Project Objectives

- Prepare a structured automation testing plan for a live e-commerce website using Playwright and JavaScript.
- Design service-wise coverage with realistic modules that can be executed safely on AJIO.
- Create traceable test coverage using an RTM with Regression Coverage to show regression coverage clearly.
- Use Page Object Model design to separate locators, actions, test data, and assertions.
- Generate professional execution evidence through Allure Report + Playwright HTML reports, screenshots, traces, and CI logs. Allure Reports will also be used to provide execution trends, historical results, categorized failures, flaky test observation, screenshots, and advanced analytics.
- Avoid unsafe areas such as real payment submission, account misuse, or handling real personal data.

Technology Stack

Technology	Purpose	Usage in Project
Playwright	Browser automation	Automates UI workflows, assertions, cross-browser execution, traces, and screenshots.
JavaScript	Scripting language	Used for writing test cases, page objects, reusable helper functions, and validations.
Node.js	Runtime environment	Runs Playwright tests and manages packages through npm.
Page Object Model	Framework pattern	Separates page locators and actions from test scripts for maintainability.
Git and GitHub	Version control	Stores project code, documentation, and workflow files in a repository.
GitHub Actions	CI execution	Runs automated tests and uploads reports after push or pull request events.
Allure Report + Playwright HTML Report	Execution evidence	Provides pass/fail status, duration, test details, and failure information.
Trace Viewer	Debugging evidence	Shows step-by-step execution, screenshots, network calls, and action timeline for failures.

Scope of Testing

Category	Description
In Scope	Homepage, navigation, product catalogue, product detail page, bag/cart behavior, offers, wishlist entry behavior, API/network observation, responsive checks, and safe checkout boundary validations.
Out of Scope	Real payment confirmation, real order placement, backend database checks, admin panel changes, scraping activities, and use of real customer personal information.
Safe Boundary	Tests can validate flow up to safe points such as product selection, bag updates, visible pricing, forms, and network responses, but must not trigger real transactions.

Services / Modules Planned

Service / Module	Coverage Focus	Cases	Severity
M1 - Homepage and Navigation	Header, menus, banners, footer links, page load checks, navigation redirects.	15	High
M2 - User Access and Session State	Login entry behavior, guest restrictions, session visibility, protected page redirection.	15	Critical
M3 - Product Discovery and Catalog Experience	Search, category browsing, filters, sorting, product cards, pagination, result consistency.	15	Critical
M4 - Product Detail Validation	PDP content, product images, price, size options, availability labels, recommendations.	15	High
M5 - Shopping Bag and Cart Management	Add/remove item, quantity handling, bag count, subtotal, persistence after refresh.	15	Critical
M6 - Wishlist and Saved Items	Guest wishlist restriction, login-gated save behavior, wishlist icon state, saved item navigation.	15	Medium
M7 - Promotions and Pricing Validation	Offer banners, discounted price display, coupon/promo visibility, price consistency.	15	High
M8 - API and Network Monitoring	Search/product/cart network responses, status codes, request parameters, response timing.	15	High
M9 - Responsive and Cross-Browser Experience	Chromium/Firefox execution, mobile/tablet view, layout integrity, tap target visibility.	15	Medium

Detailed Module Coverage

M1 - Homepage and Navigation

Validate that the homepage loads successfully, banners are visible, top navigation opens expected sections, footer links do not break, and search entry points are accessible. These checks confirm that the user can start normal shopping journeys from the landing page.

M2 - User Access and Session State

Validate safe authentication boundaries such as login page availability, invalid input validation, guest restrictions, protected wishlist/account redirection, and session indicators. Actual OTP or sensitive account operations are avoided unless a trainer-provided test account is available.

M3 - Product Discovery and Catalog Experience

Validate product browsing through categories, keyword searches, filter application, sorting behavior, visible product cards, and no-result states. Search and filtration are included here as capabilities of the catalog experience, not as separate modules.

M4 - Product Detail Validation

Validate that a product detail page displays title, image, price, size/variant options, delivery information, offers, and related product sections wherever available. Product card data and product detail data should remain consistent.

M5 - Shopping Bag and Cart Management

Validate add-to-bag actions, remove item actions, quantity updates, bag count, subtotal, empty bag state, and persistence after refreshing or returning from another page. This module represents a critical e-commerce flow.

M6 - Wishlist and Saved Items

Validate the wishlist entry points, guest access restrictions, login redirection, saved item icon behavior, and navigation to saved items. Since wishlist may require login, the plan uses safe guest and account-state checks.

M7 - Promotions and Pricing Validation

Validate offer labels, discount visibility, price formatting, coupon/promo messaging, and price consistency between catalogue, product detail, and bag views. The focus is display and calculation validation, not real payment or order creation.

M8 - API and Network Monitoring

Use Playwright response listeners and route interception to observe product/search/cart related calls. Validate status codes, request parameters, response timing, and basic payload presence without modifying production data.

M9 - Responsive and Cross-Browser Experience

Run selected flows on supported browsers and viewports. Validate layout stability, visible controls, non-overlapping content, accessible buttons, and consistent behavior across Chromium and Firefox.

Types of Testing Planned

Testing Type	Purpose in This Project
Functional Testing	Confirms that each customer flow performs the expected behavior, such as navigation, product opening, and bag updates.
Validation Testing	Checks empty input, invalid input, disabled actions, and form-level messages where applicable.
UI Assertion Testing	Verifies visible headings, buttons, prices, labels, item counts, error messages, and confirmation text.
Regression Testing	Re-runs the selected stable test suite after script updates to ensure existing workflows still pass.
Session Testing	Checks whether selected data, such as bag state, remains stable after refresh or navigation.
API / Network Testing	Observes real browser network calls and validates response status, timing, and request correctness.
Responsive Testing	Checks usability and layout behavior under mobile, tablet, and desktop viewports.
Cross-Browser Testing	Runs selected scenarios in Chromium and Firefox to detect browser-specific failures.

Automation Approach

- Use Playwright Test runner with JavaScript for structured execution and clear assertion handling.
- Create separate spec files based on service modules, not based on random feature names.
- Maintain page classes for HomePage, CatalogPage, ProductPage, BagPage, WishlistPage, OffersPage, and NetworkHelper.
- Prefer stable Playwright locators such as getByRole, getByText, getByLabel, getByPlaceholder, and visible text-based locators.
- Avoid fragile selectors such as long XPath chains and nth-child selectors unless there is no stable alternative.
- Use reusable test data files for sample keywords, product names, postal codes, and dummy user details.
- Capture screenshots and traces only where needed so the report remains useful and not overloaded.

Planned Project Structure

Folder / File	Purpose
pages/home.page.js	Homepage, navigation, header, footer, and banner actions.

pages/catalog.page.js	Product discovery, category, search, filter, and sorting actions.
pages/product.page.js	Product detail validation, image, price, size, and product actions.
pages/bag.page.js	Shopping bag, item quantity, remove item, and subtotal checks.
pages/wishlist.page.js	Wishlist entry, guest restrictions, and saved item state checks.
tests/home-navigation.spec.js	Homepage and navigation scenarios.
tests/catalog-experience.spec.js	Product discovery and catalog scenarios.
tests/bag-management.spec.js	Shopping bag and cart scenarios.
tests/network-validation.spec.js	API/network observation scenarios.
test-data/	Reusable dummy data for keywords, products, addresses, and form inputs.
playwright-report/	Generated HTML execution report after test run.

Test Data Strategy

Because AJIO is a live public platform, the project must use safe and replaceable test data. Tests must not use real customer data, real payment details, or actions that may create actual transactions. Product names, categories, and offer texts may change, so the data strategy must support easy updates.

- Use dummy names, email formats, contact details, and postal codes only where form validation is tested safely.
- Use visible product names from the live website as configurable test data, not hard-coded inside test logic.
- Use product search keywords such as shoes, kurta, jeans, bag, or watch based on visible website availability.
- Avoid depending on a single product that may become unavailable or removed from the catalogue.
- Maintain separate data files so product keywords, sizes, and validation inputs can be changed without editing every test.

Risks and Mitigation

Risk	Mitigation
Dynamic UI changes	Use user-facing locators, keep selectors in page objects, and avoid positional selectors.
Product availability changes	Use configurable product keywords and select alternate products when the catalogue changes.

Login restrictions or OTP	Avoid unsafe login dependency; validate guest restrictions unless trainer provides test credentials.
Slow public website response	Use Playwright auto-waiting, sensible timeouts, and rerun after confirming site availability.
Promotion text changes	Assert general offer visibility or price format instead of hard-coding every promotional word.
Network request changes	Observe meaningful request patterns and avoid depending on undocumented internal endpoint names.

Reporting and Evidence Strategy

- Playwright Allure Report + Playwright HTML report will be used as the primary execution report. Allure Reports will also be used to provide execution trends, historical results, categorized failures, flaky test observation, screenshots, and advanced analytics.
- Screenshots will be collected for failed scenarios and important evidence points.
- Trace files will be retained for failed cases to support step-by-step debugging.
- GitHub Actions logs will serve as CI execution evidence when tests are triggered from the repository.
- A final summary will mention total planned scenarios, automated scenarios, pass count, fail count, and pending items.

Final Deliverables

Deliverable	Description
Capstone testing report	Complete document describing scope, modules, approach, RTM, risks, and deliverables.
Test case document	Module-wise test cases with expected results, priority, status, and Regression Coverage mapping.
Playwright automation project	JavaScript test scripts organized by service/module using Page Object Model.
Test data files	Dummy and reusable data for products, keywords, addresses, and safe form validations.
Execution report	Playwright Allure Report + Playwright HTML report showing test run details and final results.
Evidence folder	Screenshots, traces, or CI logs supporting important passed and failed scenarios.

Final summary	Short summary of coverage, execution result, known limitations, and conclusion.
---------------	---

Expected Outcomes

The completed framework is expected to reduce repetitive manual verification, improve regression speed, provide better debugging evidence, and create reusable automation components for future expansion.

- Reusable Playwright automation framework using Page Object Model
- Organized module-wise test scripts with maintainable structure
- Playwright HTML Reports and Allure Reports for execution analysis
- Screenshots, traces, and execution evidence for failures and validations
- API monitoring and network validation scenarios
- CI/CD workflow integration for automated execution
- Test data files with configurable and reusable inputs
- Cross-browser and responsive execution evidence

Conclusion

This project demonstrates the implementation of a structured automation framework for a live e-commerce platform using Playwright and JavaScript. The selected modules focus on realistic customer workflows while maintaining safe testing boundaries and practical automation coverage.

The framework emphasizes maintainability through reusable components, stable locator strategies, modular design, and Page Object Model architecture. Reporting through **Playwright HTML Reports**, **Allure Reports**, screenshots, and traces improves execution visibility and debugging efficiency.

The completed automation suite is expected to reduce repetitive manual verification efforts, improve execution consistency, and strengthen overall testing reliability. Its modular approach also supports future scalability by enabling additional workflows and extended automation coverage.